นาย ณัฏฐดนย์ สาริกา 673380511-9 sec 2

🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
PC1 (VLAN 10) ----\
                   \ access
                    \
                    [S1]---- trunk ----[R1]
                    /
                   / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
Router#
Router#enable
Router#con
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

R1(config-if)#interface g0/0.10
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.10, changed state to up
encapsulation dot1Q 10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0.20
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.20, changed state to up

R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Step 2 — Switch Configuration

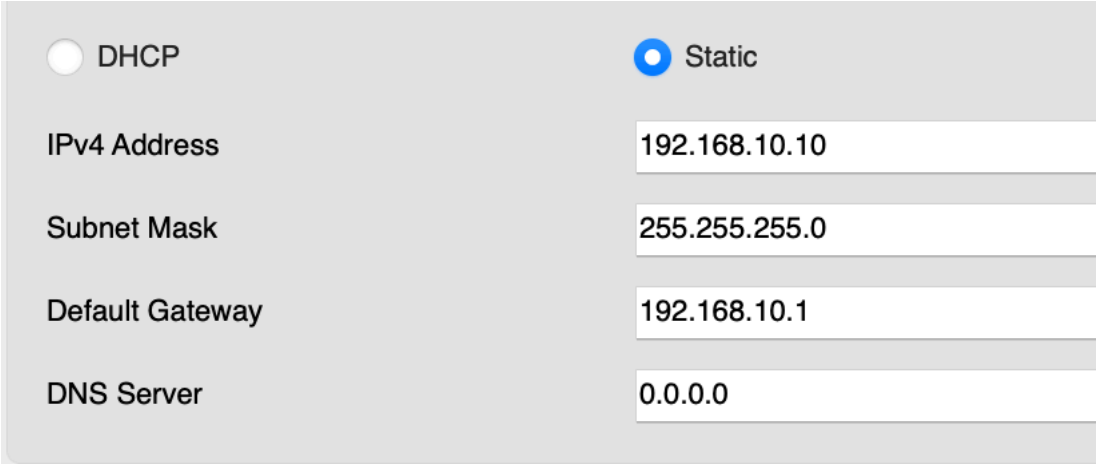
```
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
```

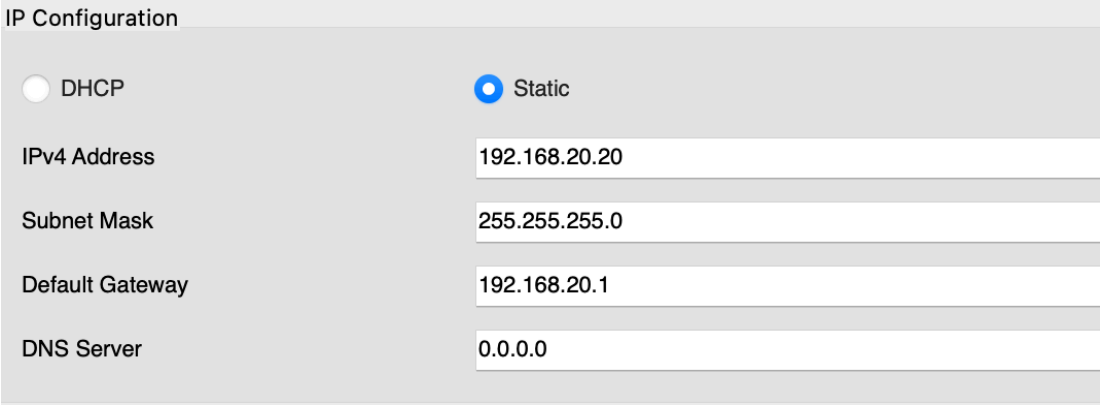
```
switchport mode access
switchport access vlan 20

end
write memory
```

```
Switch>
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

S1(config-if)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#\write memory
          ^
% Invalid input detected at '^' marker.

S1#write memory
Building configuration...
[OK]
S1#
```

Step 3 — Configure Hosts

Set static IP addresses per table.





🔍 Part 2 — Verify Connectivity

From PC1:

```
ping 192.168.20.20
```

```
Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>arp -a
  Internet Address      Physical Address      Type
  192.168.10.1          0002.1674.1101        dynamic

C:\>
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
show ip route
show ip interface brief
```

```
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1>
```

Switch checks:

```
show interfaces trunk
show mac address-table
```

```
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0002.1674.1101    DYNAMIC     Gig0/1
  10    0002.1674.1101    DYNAMIC     Gig0/1
  10    0002.1772.328d    DYNAMIC     Fa0/1
  20    0002.1674.1101    DYNAMIC     Gig0/1
  20    00e0.b0b7.7b41    DYNAMIC     Fa0/2
S1>
```

🧪 Part 3 — Deploy MIME Server

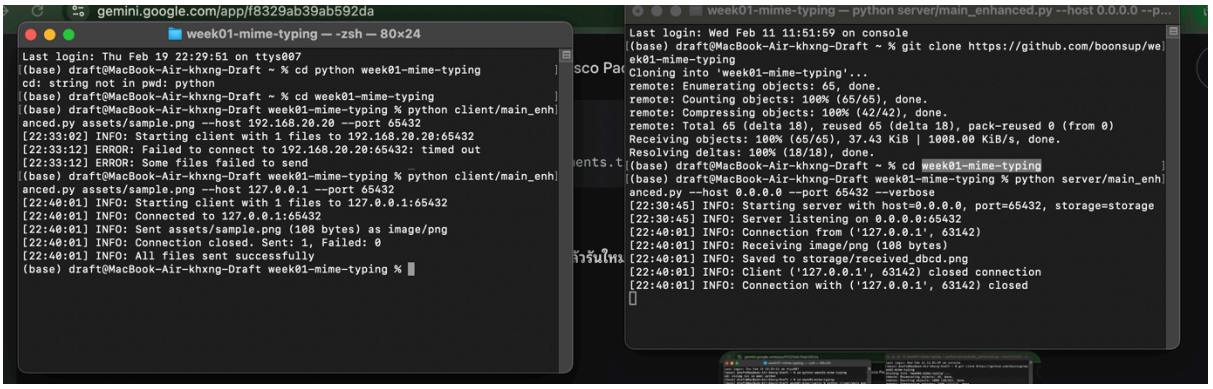
On Server (PC2)

```
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

On Client (PC1)

```
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```

Confirm file saved.



ปัญหา "Failed to connect... timed out" นั่นเป็นเพราะว่า IP 192.168.20.20 ที่ใช้ในคำสั่ง Client นั้น **เป็น IP จำลองที่อยู่ใน Packet Tracer** ซึ่งเครื่อง Mac มองไม่เห็น ต้องใช้ **Loopback IP (127.0.0.1)**

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

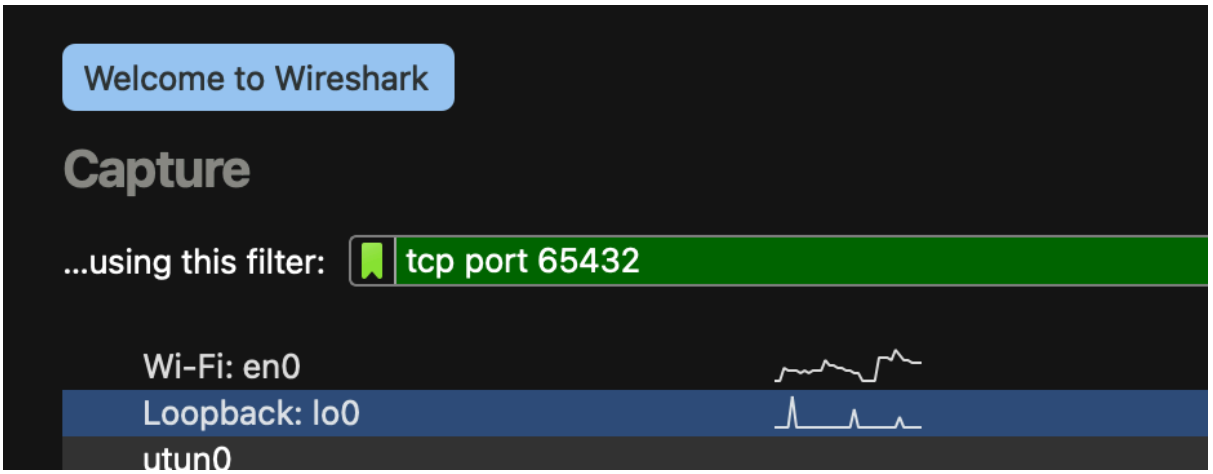
Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

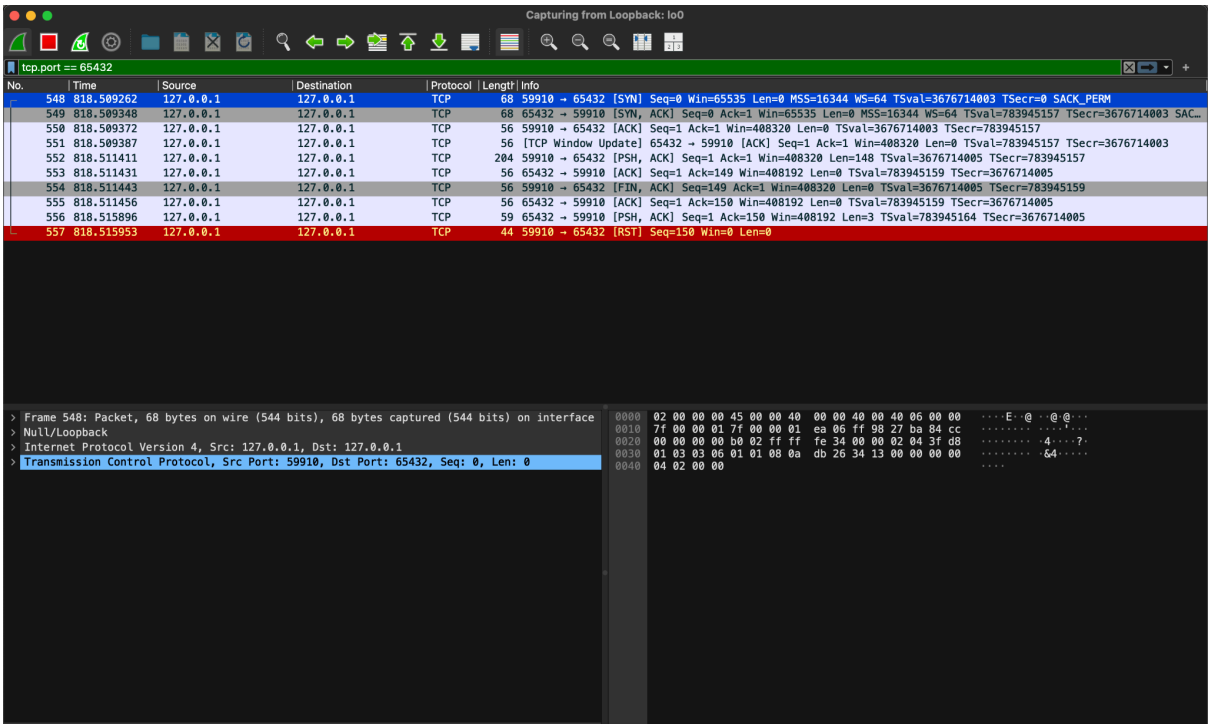
```
tcp port 65432
```



Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.



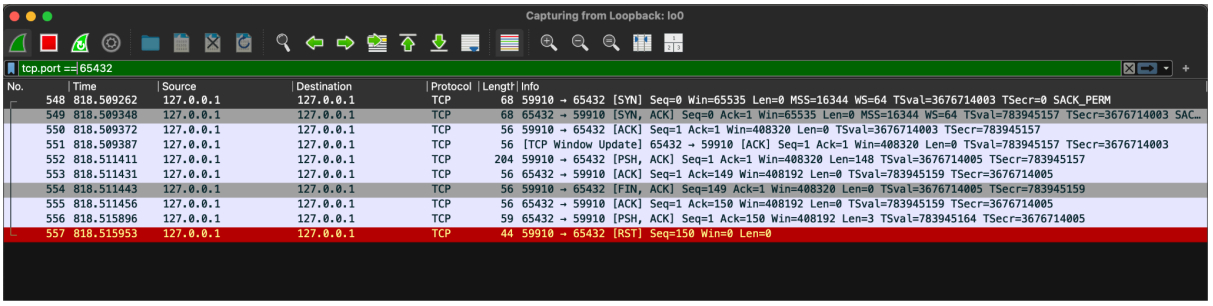
Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.



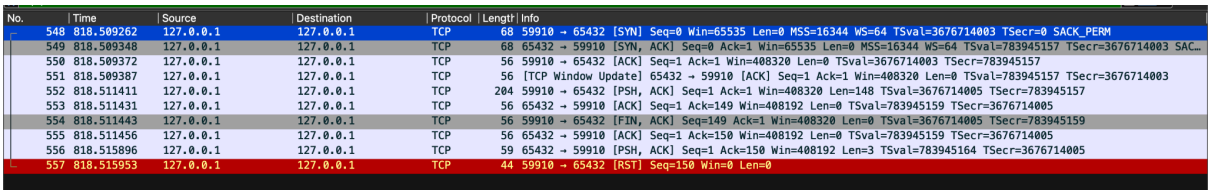
จะไม่เกิดการหา mac address เพราะใช้คอมเครื่องเดียวในการรัน คอมพิวเตอร์จะคุยกันเอง

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.



1. **SYN**: Client ส่งคำขอเชื่อมต่อ
2. **SYN, ACK**: Server ตอบรับการเชื่อมต่อ
3. **ACK**: Client ยืนยันการเชื่อมต่อครั้งสุดท้าย

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

```
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
OK
```

You should see JSON header similar to:

```
{"mime_type":"image/png","size":12345}
```

Followed by binary.

D — VLAN Tags

If capturing on trunk:

Look for:

```
802.1Q Virtual LAN
```

Note VLAN ID.

จะไม่เห็น **802.1Q (VLAN Tag)** ใน Wireshark เมื่อรัน Server และ Client บนเครื่อง Mac เครื่องเดียวกันเพราะ ข้อมูลที่วิ่งผ่าน IP `127.0.0.1` เป็นการคุยกันภายในระบบปฏิบัติการ (Software) ซึ่งไม่ได้ผ่านการประมวลผลของ Network Card หรือ Switch จริงๆ จึงไม่มีการแปะป้าย VLAN ID ใดๆ

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

```
R1>configure terminal
              ^
% Invalid input detected at '^' marker.

R1>enable
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0.10
R1(config-subif)#shutdown

R1(config-subif)#
```

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

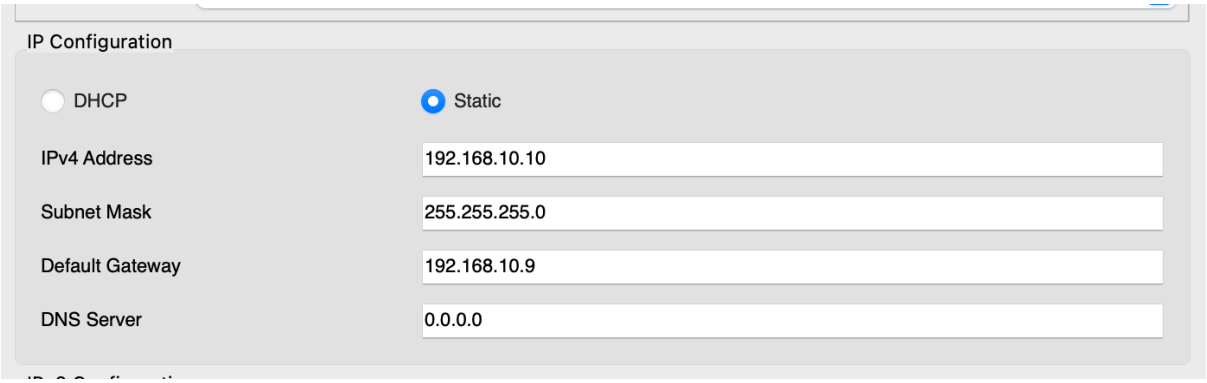
Client จะหยุดนิ่งเป็นเวลานานก่อนจะขึ้น Error Timeout และจะเห็น Client พยายามส่งแพ็กเก็ต [SYN] ซ้ำๆ หลายรอบ (Retransmission) แต่ไม่มี [SYN, ACK] ตอบกลับมาจาก Server

---

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete




```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

```
C:\>arp -a
No ARP Entries Found
C:\>
```

ไม่พบ MAC Address ของ Gateway เพราะ PC พยายามหาเครื่องที่ไม่มีอยู่จริงในระบบ

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

บน Localhost แพ็กเก็ตไม่มีทางหายเองได้ แต่หากมีแพ็กเก็ตข้อมูลสูญหายระหว่างทาง ฝั่งผู้รับ (Server) จะไม่ส่งสัญญาณตอบรับ (ACK) กลับมา

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
   - เพราะ TCP เป็นโปรโตคอลที่เน้นความน่าเชื่อถือ หากผู้ส่งไม่ได้รับสัญญาณตอบรับ (ACK) ภายในเวลาที่กำหนด (Timeout) จะถือว่าข้อมูลสูญหายและทำการส่งใหม่เพื่อให้มั่นใจว่าข้อมูลถึงปลายทางครบถ้วน
2. How does VLAN tagging affect broadcast domains?
   - การทำ VLAN Tagging (802.1Q) ช่วยแบ่ง Physical Switch ออกเป็นหลาย Logical Networks ซึ่งเป็นการแยก Broadcast Domain ออกจากกัน ทำให้ Broadcast Traffic ของ VLAN หนึ่งไม่ไปรบกวน VLAN อื่น ช่วยเพิ่มประสิทธิภาพและความปลอดภัย
3. What role does MIME play compared to TCP headers?
   - TCP Header ทำหน้าที่ควบคุมการส่งข้อมูลให้ถูกต้อง (Transport) แต่ไม่ทราบว่าข้อมูลนั้นคืออะไร ส่วน MIME ทำหน้าที่ระบุประเภทของข้อมูล (Application Metadata) เช่น `image/png` เพื่อให้แอปพลิเคชันปลายทางทราบวิธีจัดการหรือแสดงผลข้อมูลนั้นๆ ได้ถูกต้อง
4. Where does routing decision occur?
   - การตัดสินใจหาเส้นทาง (Routing Decision) เกิดขึ้นที่ Layer 3 (Network Layer) โดยในแล็บนี้เกิดขึ้นที่ Router (R1) ซึ่งใช้ Subinterfaces ในการเชื่อมต่อและส่งต่อข้อมูลระหว่าง VLAN 10 และ VLAN 20
5. Why does ARP precede TCP communication?

- เพราะก่อนที่คอมพิวเตอร์จะส่งแพ็กเก็ต IP ผ่านเครือข่าย Ethernet ได้ มันจำเป็นต้องทราบ MAC Address ของปลายทาง (หรือ Gateway) เพื่อสร้าง Frame ที่ถูกต้อง ARP จึงต้องทำงานก่อนเพื่อค้นหา MAC Address จาก IP Address ที่ระบุ

📊 Reflection Exercise

Explain:

> How does application meaning survive multiple encapsulation layers?
> - ความหมายของข้อมูล ถูกรักษาไว้ผ่านกระบวนการ Encapsulation โดยแต่ละ Layer จะเพิ่ม Header ของตัวเองเพื่อจัดการหน้าที่เฉพาะ (เช่น การระบุที่อยู่หรือความถูกต้อง) แต่จะไม่แก้ไขเนื้อหา (Payload) ด้านใน เมื่อถึงปลายทาง กระบวนการ De-encapsulation จะถอด Header ออกทีละชั้นจนเหลือเพียงข้อมูล JSON และ Binary ต้นฉบับที่แอปพลิเคชันเข้าใจ

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy
- Reliability engineering
- Metadata semantics
- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.